

Lab 3 Report

By: Sunny Kang, Tracy Yang

May 3, 2023

ECE 2049 D Term 2023

Group 3, Board 24

Introduction

Through calculating the date and time as well as the degrees in both fahrenheit and celsius, we were able to familiarize ourselves in the use and implementation of the timer, ADC values, and peripherals. We also converted decimal to ASCII and used the LCD screen to display texts. By doing so, we created a mini-time and temperature display that can be edited using the S1 and S2 buttons and the potentiometer.

Discussion and Results

For the prelab, we calculated the time from seconds to days as well as took into account the leftover seconds. To do this, we passed a copy of time into the function to lessen the processing time. Because of this, the computer does not need to continuously update as time changes. We continued this on to hours, minutes, and seconds, and then we calculated the number of months there are as well as noted the days that couldn't fit into a whole month. We used a switch case to determine and display the month name as shown in the code of figure 4. We worked on our temperature function afterwards, where we first solved for the degrees in fahrenheit and displayed both under the date and time.

Once completing the prelab, we configured the timer, where we had the max count equal to 3276, making the tick intervals change for every 1 second. This is shown below in figure 1. We determined that we should store the time count as an unsigned long as it has 16 bits, making it big enough to store a large number of seconds. Then, we initialized our date to November 20th.

```

void runTimerA2(void){
    TA2CTL = TASSEL_1 + ID_0 + MC_1;
    TA2CCR0 = 32767; //(max_count + 1)*1/f = 1 sec
    TA2CCTL0 = CCIE;
}

void stopTimerA2(void){
    TA2CTL = MC_0;
    TA2CCTL0 &= ~CCIE;
    timer_cnt = 0;
}

#pragma vector=TIMER2_A0_VECTOR
__interrupt void TimerA2_ISR(void)
{
    int j;
    float sumTemp = 0;
    float temperatureDegC;

    timer_cnt++;

    readADC();

    temperatureDegC = (float)((long)in_temp - CALADC12_15V_30C) * degC_per_bit +30.0;
    i = (timer_cnt - 30240000) % 30;
    temp[i] = temperatureDegC;

    for (j = 0; j <= 30; j++){ //sums up 30 temp values
        sumTemp += temp[j];
    }

    if (timer_cnt-30240000 <= 29)
        avgTempC = sumTemp/i;
    else
        avgTempC = sumTemp/30;

    Graphics_clearDisplay(&g_sContext);
    __no_operation();
}

```

Figure 1:

Figure 1 depicts TimerA2 configuring.

We moved onto configuring the resolution of temperature, where we determined that the reference voltage should be 1.5V through the calculations shown here:

$$Resolution = \frac{voltage}{2^N}, \text{ where } N = 12$$

$$\text{If } V = 1.5, \text{ resolution} = 0.0003662 \text{ V/bits} = 0.3662 \text{ mV/bits}$$

$$\text{If } V = 2.5, \text{ resolution} = 0.0006104 \text{ V/bits} = 0.6104 \text{ mV/bits}$$

$$Resolution = \frac{\Delta degrees}{\Delta bits \text{ of corresponding degrees}}$$

$$Resolution = \frac{85 - 30}{2565 - 2203} = 0.1519337^{\circ}\text{C}$$

We want to have the resolution of the ADC value parallel that of the timer count, so a smaller resolution is better suited for it, resulting in us choosing 1.5V as the reference voltage, 0.3662

mV/bits as the resolution in volts, and 0.1519337°C as the resolution in degrees celsius. To find the average of the last 36 seconds of temperature readings, we used circular indexing, as shown at the bottom of figure 1. This better allocates memory and reduces the space needed as well as uses less processing in order to make indexing more efficient. This is due to the fact that linear indexing takes data out of the queue and disrupts the data.

We lastly implemented Edit Mode where we used the peripherals. We mapped out the output of the scroll wheel into the different units of time by keeping track of the number of times button S1 is pressed. From doing so, we were able to modulate the variable with the number of conditions it needed to switch into and return back to the first condition. This is shown below in figure 2.

```
int pressed = 5;
int oldpressed = 0;
while(button != 0){
    swDelay(1);

    pressed += readLPButtons();

    if (pressed == oldpressed + 2){
        monthtosec = month * 2592000;
        daytosec = day * 86400;
        hrtosec = hr * 3600;
        mintosec = min * 60;
        timer_cnt = sec + mintosec + hrtosec + daytosec + monthtosec;
        button = 0;
        break;
    }
    if (pressed%5 == 0){
        //edit month
        month_per_bit = 0.00293040293;
        month = (int) (in_value * month_per_bit) + 1;
        displayTime(month, day, hr, min, sec);
    }
    if (pressed%5 == 1){
        //edit date
        day_per_bit = 0.00732600732;
        day = (int) (in_value * day_per_bit) + 1;
        displayTime(month, day, hr, min, sec);
    }
    if (pressed%5 == 2){
        hr_per_bit = 0.00586080586;
        hr = (int) (in_value * hr_per_bit);
        displayTime(month, day, hr, min, sec);
    }
    if (pressed%5 == 3){
        min_per_bit = 0.01465201465;
        min = (int) (in_value * min_per_bit);
        displayTime(month, day, hr, min, sec);
    }
    if (pressed%5 == 4){
        sec_per_bit = 0.01465201465;
        sec = (int) (in_value * sec_per_bit);
        displayTime(month, day, hr, min, sec);
    }
    oldpressed = pressed;
}
}
// end while (1)
```

Figure 2:

Figure 2 depicts the Edit Mode.

Summary and Conclusion

After successfully completing lab 3, we were able to learn how to convert seconds into time efficiently and celsius to fahrenheit, all displayed onto the LCD screen. Additionally, we were able to create a function that scrolls through these values and edit them with the change of the potentiometer input through ADC value conversion. After changing these values, we were able to update them onto the screen.

Appendix

```
void calculateTime(long unsigned int inTime){

    day = inTime / (86400) ; //int value of day (/365)
    // day = day / (60); //int value of day (/365)
    // day = day / (60); //int value of day (/365)
    inTime = inTime % (86400); //left over hours in sec

    hr = inTime / (3600); //int value of hr
    inTime = inTime % 3600; //left over minutes in sec

    min = inTime / 60; // int value of min
    inTime = inTime % 60; //left over seconds in sec

    sec = inTime;

    month = day/30;
    day = day % 30;

}
```

Figure 3:

Figure 1 depicts the calculations that convert an input of time into days, hours, minutes, seconds, and months.

```

void displayTime(unsigned long month, unsigned long day, unsigned long hr, unsigned long min, unsigned long sec){

    char dayBuffer[10];
    char hrbuffer[10];
    char minbuffer[10];
    char secbuffer[10];
    char timeBuffer[10];

    switch (month){
    case 1:
        sprintf(dayBuffer, "JAN %d", day);
        break;
    case 2:
        sprintf(dayBuffer, "FEB %d", day);
        break;
    case 3:
        sprintf(dayBuffer, "MAR %d", day);
        break;
    case 4:
        sprintf(dayBuffer, "APR %d", day);
        break;
    case 5:
        sprintf(dayBuffer, "MAY %d", day);
        break;
    case 6:
        sprintf(dayBuffer, "JUNE %d", day);
        break;
    case 7:
        sprintf(dayBuffer, "JULY %d", day);
        break;
    case 8:
        sprintf(dayBuffer, "AUG %d", day);
        break;
    case 9:
        sprintf(dayBuffer, "SEPT %d", day);
        break;
    case 10:
        sprintf(dayBuffer, "OCT %d", day);
        break;
    case 11:
        sprintf(dayBuffer, "NOV %d", day);
        break;
    case 12:
        sprintf(dayBuffer, "DEC %d", day);
        break;
    }

    sprintf(hrbuffer, "%d", hr);
    sprintf(minbuffer, "%d", min);
    sprintf(secbuffer, "%d", sec);
    sprintf(timeBuffer, "%s:%s:%s", hrbuffer, minbuffer, secbuffer);

    Graphics_drawStringCentered(&g_sContext, dayBuffer, AUTO_STRING_LENGTH, 48, 25, TRANSPARENT_TEXT);
    Graphics_drawStringCentered(&g_sContext, timeBuffer, AUTO_STRING_LENGTH, 48, 43, TRANSPARENT_TEXT);
    Graphics_flushBuffer(&g_sContext);
}

```

Figure 4:

Figure 2 depicts the switch statements that go between what month it is as well as displays all of the time, formatted so that it follows the prescribed format.

```

void displayTemp(float inAvgTempC){
    int tempFint = (int)(inAvgTempC * (1.8) + 32);
    int tempFdec = abs((int)((inAvgTempC * (1.8) + 32)*10) % 10);
    int tempCint = (int) inAvgTempC;
    int tempCdec = abs((int) (inAvgTempC*10) % 10);

    char bufferTempC[10];
    char bufferTempF[10];

    sprintf(bufferTempC, "%.1dC", tempCint, tempCdec);
    sprintf(bufferTempF, "%.1dF", tempFint, tempFdec);

    Graphics_drawStringCentered(&g_sContext, bufferTempC, AUTO_STRING_LENGTH, 48, 61, TRANSPARENT_TEXT);
    Graphics_drawStringCentered(&g_sContext, bufferTempF, AUTO_STRING_LENGTH, 48, 79, TRANSPARENT_TEXT);
    Graphics_flushBuffer(&g_sContext);
}

```

Figure 5:

Figure 3 shows the code that converts temperature in celsius to that in fahrenheit as well as displays it onto the LCD where celsius is under the time and fahrenheit is under celsius and is formatted as prescribed.